



**User manual for running MicroICS analysis
pipeline
(version 2)**

Table of contents

Quick start.....	4
1. Image requirements	7
2. Computer requirements	8
2.1. Setup procedure for Windows systems.....	9
2.1.1. Install Docker.....	9
2.2.2. Building the Docker image	9
3. Preparation of the data for running the MicroICS pipeline	10
6. Generating features using the MicroICS pipeline	12
7. Machine learning benchmarking	13
5.1. Running the benchmark with default settings.....	14
5.2. Running the benchmark with all available models.....	14
5.3. Selecting specific models	14
5.4. Understanding the benchmark output.....	16
8. Inference — predicting biological labels for new images	18
6.1. Build and save the models	18
6.2. Calculate features on new images	19
6.3. Run inference on new images	20
6.2. Understanding the inference output.....	20
9. MicroICS implementation examples	25
7.1. How do you use the pipeline if you want to link biofilm features to properties other than the strain name?	25
7.2. Using the pipeline to create images with reduced number of layers and determining whether the number of layers in a 3D image affects classifier performance.....	25
10. Feature descriptions to help with interpretation	26

Introduction

MicroICS is an open-source framework for the analysis of 3D biofilm images. It consists of three independent but interoperable modules: feature extraction from 3D biofilm images, machine learning-based classification, and inference – applying a trained model to new, unseen images. Each module can be run on its own, and the classification module accepts features generated by other tools, so MicroICS can be combined with existing quantitative image analysis software such as BiofilmQ.

Machine learning is not the right approach for every biological question. The following may help you decide if MicroICS is likely to help you:

You can label your images. A label is simply the property you want the model to learn – the strain name, the treatment, the isolation source, or anything else connected to your question. Without labels there is nothing for the model to learn from.

The property you care about plausibly appears in biofilm structure. If it leaves no structural trace, no algorithm will find it. If you are not sure, this is itself worth testing: given enough representative data, a model that cannot be trained to useful accuracy is telling you something – either the signal is not in the structure, or your dataset is not yet able to reveal it.

You have enough replicates, both biological and technical. This requirement is the one most often underestimated. Our own dataset required substantial technical replication per condition before reaching useful accuracy. Image every group across several independent imaging sessions. Repeating the experiment on different days is what makes a conclusion firm – a model trained on a single session may simply have learned that session.

You have more images than you can reasonably inspect by eye, and more measurements than you can sensibly test one at a time.

MicroICS will not help if:

You have only a handful of images per group. The model will memorise them rather than learn from them.

You want to know why, biologically. The model will tell you which structural features matter for your question. Interpreting why they matter remains your job – and is where the biology begins.

How this manual is organised

The next section, Quick start, gives the key decisions and the commands that follow from them – enough to run the pipeline without reading further.

The chapters after that are arranged sequentially, taking you from setup and data preparation through feature extraction, model benchmarking, and inference on new images. They are written for users in biology and related fields, without assuming specialist computational expertise.

Quick start

If you don't want to read the whole manual, follow the five steps below, then run the commands for your path. Each step links to the chapter that explains it in full.

Step 1 — Decide your question, and label your images

The label is your question. The label attached to each image is what the model learns to distinguish. Change the label and you change the biological question – the pipeline itself does not change.

Labels are read from the filename, from the string following `_st_`. To ask a different question, replace that string with your property of interest.

For example, to link biofilm structure to clinical relevance rather than strain identity, replace the strain name with CI (clinically relevant) or CNI (not clinically relevant):

```
13042023_s_Lm_st_L628_p_C06_pos001_tm_24_ch_Syto9_z_21
```

```
13042023_s_Lm_st_CI_p_C06_pos001_tm_24_ch_Syto9_z_21
```

```
13062023_s_Lm_st_L1764_p_G05_pos002_tm_24_ch_Syto9_z_21
```

```
13062023_s_Lm_st_CNI_p_G05_pos002_tm_24_ch_Syto9_z_21
```

Everything else in the filename stays the same – the date in particular is used for the batch-effect check.

Details explained in [Image requirements](#)

Step 2 — Prepare your images

Images must be 3D TIFF stacks, single channel, with consistent acquisition settings and voxel dimensions across the whole dataset. Most microscope formats (.czi, .lif, .nd2) can be exported to TIFF from Fiji/ImageJ via Bio-Formats.

Details explained in [Image requirements](#)

Step 3 — Download the code, install Docker, build the image

Install Docker Desktop (Windows, macOS) or Docker Engine (Linux).

Download or clone the repository:

```
git clone https://github.com/SkBlaz/biofilm-classification
```

Build the MicroICS image:

```
docker compose build --no-cache
```

Details explained [Computer requirements](#)

Step 4 — Create your configuration file

MicroICS needs to know where your images are and where to write results. Create five folders – two for images (training, inference) and three for outputs (training results, inference features, inference results) — and list their paths in a configuration file inside the repository folder. The examples in this manual use a file named `test_paths.txt`, which is specified explicitly in every command with `--env-file`.

Windows users: paths must be converted to Docker format – `C:\Users\me\images` becomes `/c/Users/me/images`.

Details explained in [Preparation of the data for running the MicroICS pipeline](#)

Step 5 — Choose your analysis path

The workflows below are quick-reference command sequences, one for each task you might want to perform. Each links to the chapter that explains the command, its outputs, and how to interpret them – read those chapters before running the pipeline on your own data for the first time.

Bear in mind that a model can only read the type of feature table it was trained on. If you trained on MicroICS plus features from another tool, you must supply MicroICS plus features from the same tool at inference, with identical settings. Workflow B pairs with workflow D; workflow C pairs with workflow E.

My goal is:	workflow
Get structural measurements out of my 3D images, nothing more.	A
Train a model to tell my biological groups apart, using MicroICS features.	B
Train a model using MicroICS features plus features from another tool (BiofilmQ, COMSTAT, your own).	C
Predict on new images, using a model trained via workflow B.	D
Predict on new images, using a model trained via workflow C.	E

Workflow A – extract features:

```
docker compose --env-file test_paths.txt run --rm image 1 datafile.tsv 10
generate_features
```

Details explained in [Generating features using the MicroICS pipeline](#)

Workflow B – extract, then train model on MicroICS features:

Run workflow A first, then train on the resulting `datafile.tsv`:

```
docker compose --env-file test_paths.txt run --rm image 1 datafile.tsv 10
learning_benchmark_save_models --all_learners
```

Details explained in [Machine learning benchmarking](#) and [Inference — predicting biological labels for new images](#)

Workflow C – extract, add external features, then train the model:

Run Path A first, then add your external features to `datafile.tsv` using your own script.

Before training, check that the data file is formatted correctly:

- the label column is last

- all added values are numeric
- there are no missing values
- the file is saved as a TSV

The command is identical to Path B – what differs is the contents of the datafile, not the command:

```
docker compose --env-file test_paths.txt run --rm imagine 1 datafile.tsv 10
learning_benchmark_save_models --all_learners
```

Details explained in [Machine learning benchmarking](#) and [Inference — predicting biological labels for new images](#)

Workflow D – predict with a workflow B model:

Extract features from your unknown images:

```
docker compose --env-file experiment_inference.txt run --rm imagine 4
unknown_features.tsv 10 generate_features --unlabeled
```

Then run inference:

```
docker compose run --rm imagine 4 unknown_features.tsv 10 inference
```

Details explained in [Inference — predicting biological labels for new images](#)

Workflow E – predict with a workflow C model

Extract features from your unknown images as in Path D, then add the same external features you used in training, prepared in the same way. Then run inference:

```
docker compose run --rm imagine 4 unknown_features.tsv 10 inference
```

Details explained in [Inference — predicting biological labels for new images](#)

1. Image requirements

MicroICS requires 3D biofilm images (z-stacks) as input, with each image comprising multiple layers representing different depths through the biofilm. Images must be saved in TIFF format, as other formats are not currently compatible with the pipeline.

Each input file must correspond to a single imaging position and include all z-layers for that position as a single multi-page TIFF stack. The method for creating this format will depend on your microscope and acquisition software. Two common scenarios are described below:

Zeiss microscopes (using positions and hardware auto-focus as described in paper): When imaging multiple positions in a microtiter plate, Zeiss microscopes may save all positions in a single file. These must be split into individual TIFF files before analysis – one file per position, containing all z-layers. This can be achieved using the open-source software ImageJ/Fiji (<https://imagej.net/software/fiji/>) with the provided ImageJ script for splitting and converting LSM files (see Figure 1).

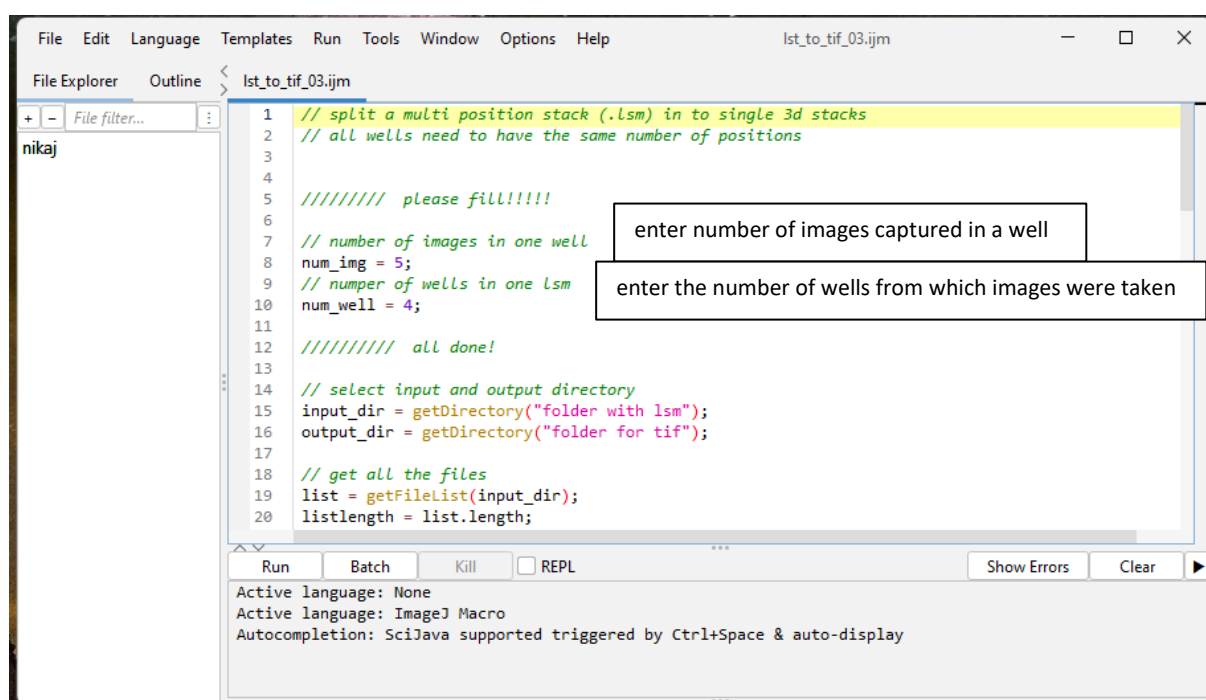


Figure 1. A screenshot of the script "lsm_to_tif," available at 10.5281/zenodo.20377151, is shown. The script enables the conversion of a lsm file with several 3D stacks taken at different positions in microtiter plate into a single 3D stack per position (one-position--one-set of images – one 3D stack) in tif format. The script is loaded by dragging it onto the ImageJ; once it opens, the number of images per well and the number of wells in which images are captured must be adjusted. This script also relies on the image naming shown below. If you do not follow these, the script will not work.

Other microscopes and acquisition systems: If your microscope saves images directly as individual TIFF stacks – one file per position – no conversion is necessary, provided the files meet the naming conventions described in Section 2.2. If your microscope uses a different proprietary format, ImageJ/Fiji supports a wide range of formats through the Bio-Formats plugin and can be used to export images as multi-page TIFF stacks before analysis.

The execution of the MicroICS pipeline depends heavily on the information contained in the image names, which should follow the example below.

Scheme for image naming:

<date>_s_<species>_st_<strain>_p_<well_coordinates>_pos<well_position>_tm_<growth_time>_ch_<channel>_z_<layer_distance>

Example of an image name:

07112023_s_Lm_st_L1823_p_F02_pos005_tm_24_ch_Syto9_z_21

Explanation of the name:

- 07112023: Date currently included in DayMonthYear to make it easier to track results
- s_Lm: s_ indicates the species, Lm means *Listeria monocytogenes*
- st_L1823: st_ denotes the strain, L1823 is the name of the strain (strain identity)
- p_F02: p_ denotes the well coordinates on the 96-well plate, F02 is the actual position on the 96-well plate
- pos005: pos denotes the measured position within the individual well, e.g., position 5 in well F02
- tm_24: tm_ stands for the measurement time, 24 means that the images were taken after 24 hours of biofilm growth
- ch_Syto9: ch_ denotes the channel, Syto9 is the name of the dye and filter used
- z_21: z_ denotes the number of layers in the 3D image, which also reflects the distance between two consecutive layers.

The most important parts of the naming structure are: Date, _s_, _st_, _p_, _pos. The names are parsed on the back-end according to these patterns. Any deviation from the exact patterns listed will result in failed or incorrect processing of the data.

The renaming process can be carried out semi-automatically by the software, e.g. <https://www.advancedrenamer.com/>, or by user-defined scripts. Examples and the user manual are available here https://www.advancedrenamer.com/user_guide/v3/complete_guide together with graphical representations.

Before proceeding, ensure your dataset contains a sufficient and approximately equal number of images per class. The machine learning benchmark requires enough images in each class to split the dataset into training and prediction subsets. An approximate guide is provided in Section 5.

2. Computer requirements

The pipeline is designed to run on both Windows and Unix-based systems, or more generally on any system that supports Docker. Hardware requirements include at least two computing cores (CPUs) and 4 GB of RAM. The files required to run the pipeline are approximately 500 MB, and at least 1 GB of free disk space is recommended for smooth analysis.

If you follow this manual step by step and copy the commands exactly as provided, the pipeline should run without issues on a correctly configured system. Most problems users encounter stem from the operating system environment itself, such as Docker not running in the background, insufficient permissions, or path formatting errors, rather than from the pipeline.

Required user level

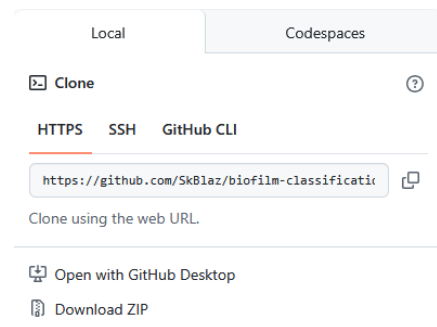
No programming experience is necessary. You will need:

- Basic familiarity with navigating folders and files in Windows or your operating system
- Basic familiarity with entering commands in the Windows command line or terminal – an introduction is available at: <https://www.geeksforgeeks.org/most-useful-cmd-commands-in-windows/>
- Basic familiarity with Docker Desktop – an introduction is available at: <https://www.docker.com/blog/getting-started-with-docker-desktop/>

2.1. Setup procedure for Windows systems

2.1.1. Install Docker

1. Download and install the Docker Desktop application from the website. This might take a minute, depending on your system and will require you to restart your computer: <https://docs.docker.com/desktop/install/windows-install/>
2. Start Docker Desktop by double-clicking on the Docker icon in the Start menu. It might run in the background and you might need to bring it to the front from the tray (bottom right corner).
3. Download "biofilm-classification-main.zip" from the github <https://github.com/SkBlaz/biofilm-classification> :
 - a. Go to "code"
 - b. Click on "Download zip"
 - c. Save the zip file to Desktop.
4. Extract "biofilm-classification-main.zip" into the biofilm-classification-main (you can rename it if you want) folder on your desktop.



2.2.2. Building the Docker image

1. Navigate to the folder where you extracted the zip file, right-click, and select "Open in Terminal" to start the terminal in the "biofilm-classification-main" folder. Link to "open in terminal" can be hidden also in "show more options". If these options are not available on your system, we suggest extracting the downloaded file onto C:/, open cmd and navigate to the folder using command cd e.g., cd C:/path/to/my/folder. Cmd can be found by typing "cmd" in Windows search and is run by clicking on "Command Prompt".
2. Connect to the internet prior to running the build command.
3. Write or copy the following command into the terminal to build a Docker image specifically created for running these analyses (the -no-cache parameters ensures a clean build from scratch):

```
docker compose build --no-cache
```

Once the Docker image is built (it takes a while), you can run analyses as described below.

The Docker image will not need to be rebuilt unless changes are made to the scripts prior to the next use of Docker.

Once Docker Desktop is running, you do not need to keep the application window open. You can close the Docker Desktop window and it will continue running in the background; a small Docker icon in the taskbar confirms that it is still active and ready to use. If you accidentally close Docker completely, double-click the Docker Desktop icon to start it again. Similarly, you can close the terminal window at any time if you do not intend to run further commands, or leave it open if you plan to continue working with the MicroICS pipeline.

3. Preparation of the data for running the MicroICS pipeline

1. MicroICS needs five folders: two for images, three for outputs. Create them and note the full path of each.

Image folders – images are copied in these folders:

Folder	Contents	Example
Training images	3D TIFFs used to train the model	C:\Users\my_name\Desktop\training_images
Inference images	New 3D TIFFs you want to classify	C:\Users\my_name\Desktop\images_to_classify

Output folders — MicroICS writes here:

Folder	Contents	Example
Training results	Features of the training images, and the trained models	C:\Users\my_name\Desktop\training_results
Inference features	Features calculated from the unknown images	C:\Users\my_name\Desktop\inference_features
Inference results	Predicted labels and confidence scores	C:\Users\my_name\Desktop\inference_results

The three output folders can be left empty — MicroICS fills them. Keep them separate from your image folders so outputs are never mixed with inputs.

2. Check your voxel dimensions

MicroICS calculates structural features from the physical size of each voxel — the three-dimensional unit of your image stack, defined by the pixel size in x and y and the z-step between layers. If these values are wrong, your features will be calculated in the wrong physical units.

Find your voxel dimensions in your microscopy software metadata, or in Fiji/ImageJ via Image > Properties.

The MicroICS defaults are:

`voxel_size = 0.13 * 0.13 * 0.5`

`pixel_area = 0.13 ** 2`

If your values match, continue to 3. If they differ, open `biofilm-classification-main/src`, locate the feature generator script (search for # constants), and update both values.

If you edit this file you must rebuild the Docker image — see Section 2.2.2.

3. Prepare configuration file

MicroICS reads your folder locations from a configuration file (env file). This must be set up before running the pipeline.

Option A — a file named .env

Copy env.example in the biofilm-classification-main folder and rename the copy to .env. Docker finds this file automatically, so you never name it on the command line.

Option B — a named file, e.g. test_paths.txt (recommended)

Create a plain text file in the same folder. You must then name it in every command with --env-file test_paths.txt.

Option B is recommended because you can keep several configuration files and switch between them simply by changing --env-file. This is why the commands throughout this manual name a file explicitly. The worked examples use test_paths.txt.

4. Convert your paths to Docker format

Windows users — file paths must be converted before being entered:

replace \ with /

remove the : after the drive letter

add / at the start

C:\Users\my_name\Desktop\training_images -> /c/Users/my_name/Desktop/training_images

macOS and Linux users — your paths already work as they are (e.g. /home/my_name/training_images). No conversion is needed.

5. Fill in the configuration file

Copy the template below and replace each path with your own, following the format rules in Step 4.

Do not put spaces before or after the = sign.

Do not put quotation marks around the paths.

Lines beginning with # are comments — they are ignored by the software, and you can add your own.

```
# ----- TRAINING -----
# Folder containing the .tif images used to train the model
IMAGE_IMAGES=/c/Users/my_name/Desktop/training_images

# Folder where features of the training images and the trained models are written
IMAGE_RESULTS=/c/Users/my_name/Desktop/training_result

# ----- INFERENCE -----
# Folder containing the new .tif images you want to classify
IMAGE_INFERENCE_INPUTS=/c/Users/my_name/Desktop/images_to_classify
# Folder where features calculated from the unknown images are written
# NOTE: this is a FOLDER, not a file, despite the variable name
IMAGE_INFERENCE_DATAFILE=/c/Users/my_name/Desktop/inference_features
# Folder where the predictions are written
IMAGE_INFERENCE_OUTPUTS=/c/Users/my_name/Desktop/inference_results
```

6. Generating features using the MicroICS pipeline

Feature extraction is the first in the MicroICS pipeline. In this step, the software processes each 3D biofilm image in your input folder and calculates approximately 2,700 quantitative structural descriptors, as described at the end of the document and in Table 2. The resulting feature table is saved to your results folder and serves as the input for all subsequent steps, including machine learning benchmarking and inference.

Before running feature extraction, ensure that:

- Your input image folder is prepared and contains images in the correct TIFF format and naming convention (see Section 2).
- Your results folder exists and the paths are correctly set in your configuration file (see Section 3).
- Your voxel dimensions are correctly configured if they differ from the defaults (see Section 3, Step 3).

Running feature extraction

To run the feature extraction command, navigate to the biofilm-classification-main folder (or the name you chose when unzipping the downloaded file) and open a terminal in this folder by right-clicking and selecting 'Open in Terminal'. The Docker daemon must also be running; you should see the Docker icon in the taskbar. If Docker is not running, open Docker Desktop as described in Section 2.

To generate features run the following commands in the terminal:

```
docker compose run --rm image 1 datafile.tsv 10 generate_features
```

Alternatively, if you use your own configuration file with paths:

```
docker compose --env-file test_paths.txt run --rm image 1 datafile.tsv 10 generate_features
```

If you are on Workflow C from the Quick guide, stop here and add your external features to datafile.tsv before continuing, using the same tool and the same settings as in training.

The parameter “1” determines the parallelization rate — a higher parallelization (should not exceed the number of CPU cores in your computer) works faster, but consumes more resources of the computer, while a lower parallelization (1) works slower, but should also be completed on machines with fewer resources. This task may take quite long for large amounts of images.

The parameter “10” determines how many of the top features are used in the visualisation later in the process, but it is not currently used.

The main output of this step is datafile.tsv, where all calculated features are available for review.

Once the process is complete, check the results folder and in particular the datafile.

The file `datafile.tsv` can be opened with Excel for checking. In the first column of the data file, all analysed images should be listed by name. If some images are missing, check the format of the missing images.

An example of what kind of results you can expect can be found in the `biofilm-classification-main/examples/results_example` folder. The name of the file is `data.tsv`.

How features are calculated is described in the last chapter of the manual and in the attached table. Further details can be found in the code itself. Many examples are also given in the manuscript on description of MicroICS based on the use case of *Listeria* biofilms.

The features from the `datafile.tsv` are now prepared for running the comparison of classification models.

Important: If you need to inspect or modify the TSV feature file, for example by adding new features calculated by other software, do not open and save it directly in Microsoft Excel. Excel will silently convert the file to a plain text format, removing the TSV structure and causing errors in downstream processing. Instead, use a text editor (such as Notepad, VS Code, or similar), or open the file in Excel using the Data > Import from Text/CSV function without resaving. If you need to edit the file in Excel, save it as a plain text file (`.txt`), then convert it back to TSV format using the following command in the terminal:

```
cp datafile.txt datafile.tsv
```

7. Machine learning benchmarking

This section compares machine learning classifiers on your dataset. For each classifier, the software randomly divides your image dataset into a training set and a prediction set. The training set is used to build the classifier, and the prediction set is used to evaluate how accurately the biological labels – for example, strain identity – can be predicted from the trained model. This process is repeated for each classifier in turn, allowing direct performance comparison across methods. The result indicates which classifier is most suitable for your specific dataset and biological question.

If your goal is to classify new, previously unseen images (inference), you may skip the commands in this section and proceed directly to Section 6. Section 6 builds the model and runs inference. This section is intended primarily for users who wish to explore and compare classifier performance before selecting a specific model. However, it also provides useful background on how the models are built and evaluated, which is relevant for inference users as well. The commands for running inference on new images are provided in Section 6.

For the benchmark to run successfully, each class must contain enough images to allow the dataset to be split into training and prediction subsets in a statistically meaningful way. If any class contains too few images, the software will raise an error rather than produce unreliable results. Additionally, for the classifier to learn effectively, the number of images per class should be approximately equal – large imbalances between classes can bias the model towards the more represented class and reduce prediction accuracy for underrepresented

groups. If your dataset is imbalanced, we recommend acquiring additional images for underrepresented classes.

5.1. Running the benchmark with default settings

By default, the benchmark runs random forest only. This is the recommended starting point for most users, as random forest offers strong predictive performance, interpretable feature importance scores, and runs efficiently on standard hardware without requiring additional configuration.

The command to put in the terminal for comparison of the classifiers (benchmark) is as follows:

```
docker compose run --rm image 1 datafile.tsv 10 learning_benchmark
```

The parameter “1” determines the parallelization rate — a higher parallelization (should not exceed the number of CPU cores in your computer) works faster, but consumes more resources of the computer, while a lower parallelization (1) works slower, but should also be completed on machines with fewer resources. The parameter “10” determines how many of the top features are used in the visualisation later in the process, but it is not currently used.

Alternatively, if you use your own configuration file with paths:

```
docker compose --env-file test_paths.txt run --rm image 1 datafile.tsv 10 learning_benchmark
```

5.2. Running the benchmark with all available models

To compare all active models, such as logistic regression, XGBoost, AutoGluon, and others, you can add a single flag to the command. To run all enabled models, add the `--all_learners` parameter at the end of the run command, for example:

```
docker compose run --rm image 1 datafile.tsv 10 learning_benchmark --all_learners
```

Alternatively, if you use your own configuration file with paths:

```
docker compose --env-file test_paths.txt run --rm image 1 datafile.tsv 10 learning_benchmark --all_learners
```

This parameter modifies the function of the `learning_benchmark` and `learning_benchmark_save_models` commands.

Note that AutoGluon and TPOT are computationally demanding and may require considerably more memory and processing time than the other models. We recommend using this option only if you have access to sufficient computing resources. Please refer to section 5.3 for details.

5.3. Selecting specific models

If you wish to run a subset of classifiers – for example, to exclude computationally demanding models while still comparing more than one algorithm – individual models can be enabled or

disabled by editing `src/feature_ranking_lite.py` as described below. Some classifiers, most notably AutoGluon and TPOT, require substantially more memory than others and may cause the container to crash on standard laptops, appearing as a terminated task in the terminal with no output saved for that model. **The software is designed to store intermediate results as it runs, so if a crash occurs, most of the computation will not be lost and the software should resume from where it stopped if restarted.**

If the software crashes repeatedly for a particular model, try closing other applications to free up available memory and restart the run. If the problem persists, disable the problematic model using the instructions below and rerun. We recommend avoiding AutoGluon and TPOT unless you have access to a computer with sufficient memory and processing power.

To enable or disable these models, open `src/feature_ranking_lite.py` in a plain text editor and locate the model definition section shown below. This section contains lines similar to the following example:

```
if all_learners:
    models = {
        'dummy': DummyClassifier(),
        'decisiontree': tree.DecisionTreeClassifier(),
        'logistic': LogisticRegression(),
        'rf': tuned_rf,
        'xgb': XGBClassifier(n_estimators=100, max_depth=3, learning_rate=1, objective='binary:logistic'),
        'gridsearch': GridSearchCV(KNeighborsClassifier(), parameters, n_jobs=PARALLELISM),
        # 'tpot': TPOTClassifier(generations=5, population_size=20, cv=5, random_state=42, verbosity=2, n_jobs=PARALLELISM, memory='auto'),
    }

    # Add TPOT only if available
    if TPOT_AVAILABLE:
        #models['tpot'] = TPOTClassifier(generations=5, population_size=20, cv=5, random_state=42, verbosity=2, n_jobs=PARALLELISM, memory='auto')
else:
    # Default behavior: only RandomForest (fast)
    models = {
        'rf': tuned_rf,
    }

    # Add autogluon model only if available
    if AUTOGLUON_AVAILABLE:
        #models['autogluon'] = TabularPredictor(label="label")
```

Models that are currently disabled are marked with a `#` at the beginning of the line (this is called a comment – the software ignores any line starting with `#`). To enable a model, remove the `#` at the start of the relevant line. To disable a model, add a `#` at the start of the line. For example, in the example below, you need to add a `#` sign in front of two parts of the code to disable TPOT and AutoGluon.

`# Add TPOT only if available`

`#if TPOT_AVAILABLE:`

`#models['tpot'] = TPOTClassifier(generations=5, population_size=20, cv=5, random_state=42, verbosity=2, n_jobs=PARALLELISM, memory='auto')`

and

`# Add autogluon model only if available`

`#if AUTOGLUON_AVAILABLE:`

`#models['autogluon'] = TabularPredictor(label="label")`

After such modification in the selected models, the change should be rebuilt in the image with (this command builds the change into the existing image and not the entire image once again and takes less time, thus the `-no-cache` parameter **isn't used**):

docker compose build image

After running this command, you can proceed with the commands in the `learning_benchmark` section or continue to inference in section 6.

5.4. Understanding the benchmark output

Each benchmark runs produce the same set of output files, listed below. This section explains what each file contains and how to interpret the results.

Before describing the individual files, two terms used in the output file names require clarification:

- **Label** – the biological property the classifier was trained to predict. In the default configuration, this is strain identity, corresponding to the string that follows "st" in the image file name (see Section 2, Naming conventions). Users who wish to define a different classification target — for example, a treatment condition or phenotypic group — can do so by following the instructions in Section 7.1.
- **Date** – the date on which the experiment was performed, as indicated in the image file name. Date is used as a secondary classification target to identify features that vary with experimental batch rather than with the biological label of interest, allowing these to be separated during interpretation (see Section 5.4.3).

Output file names:

- **Classification_all** - Classification results using the full feature set. Calculated for all tested algorithms.
- **Classification_no_counts_features** - Classification results with count-based features excluded. Calculated for all tested algorithms.
- **Ablation_ranking_all** – Shows how classification performance changes as the number of features is reduced, enabling identification of the minimum feature set required for optimal accuracy. Calculated for random forest only.
- **Rankings_date** – Feature importance rankings for the date prediction task, indicating which structural features are associated with the experimental date rather than the biological label of interest. These rankings are used to identify and exclude potential batch effects from biological interpretation. Calculated for random forest only.
- **Rankings_label** – Feature importance rankings for the biological label prediction task, such as strain identity, indicating which structural features are most informative for distinguishing between the biological groups of interest. Calculated for random forest only. Detailed explanations of each output file are provided in the sections below.

Classification_all and classification_no_counts_features

These two files summarize the results of classification by selected machine learning algorithms. The columns from left to right are:

- **Tag** - has value RESULT and it has been used for parsing, but is currently not active
- **model** - type of classifier, random forest, etc. and dummy classifier as sanity test
- **upsampling** - has value 1
- **n_components** - number of input features

The classifiers can be provided with all features, currently 2700, or the number of features can be reduced in advance using the SVD decomposition model, which creates components as linear combinations of the original features; it does not select

individual original features as feature ranking does. Here, we reduce the number of features by half. When comparing the accuracy for the reduced data, you should bear in mind that the features selected by SVD differ across dimensions, meaning the model may be provided with different features and therefore produce different results.

- fold – number of iterations (repetitions)
- accuracy - number of correct predictions (key result of the comparison)
- test_set - name of the strains (labels) used for the classification test,
- thr_features – TRUE if features with threshold are included and FALSE if features with threshold are excluded; for each number of n_components result for TRUE and FALSE are given; for n_components 580 we have only FALSE (all features except the one with threshold) and for 2700 only TRUE (because all features include the one with threshold)

The following files contain information derived exclusively from classifications based on **random forest**.

Ablation_ranking

The report shows how model accuracy changes as features are added incrementally. Determining the highest accuracy for a dataset shows how many features are required to classify the strains. For interpretation, we recommend selecting the number of features that gives the highest accuracy and then using the same number of features with the highest score to explain which ones contribute the most to the classification.

For example, in ablation_ranking we find that the number of features with the highest accuracy is 81; then we go to the ranking file and select 81 features with the highest score for further interpretation.

Rankings_label and rankings_date (*Here, label refers to the string that follows _st_ in the image name, which in this case is the strain name. In classification, we refer to the property being predicted as the target, so the target here is either the label (strain) or the date.*)

For each feature calculated, the strength of its relationship to the target is indicated, with larger values indicating that the feature was more important in predicting strain and vice versa (rankings_label). We also provide rankings for date prediction (rankings_date) to account for features that may vary as a consequence of repeating the experiment on different days (batch effect is likely due to the methodology or variability of the biological system under study). This is crucial when we provide multiple images from different days (biological replicates) for a single strain to the model, as the variation within a single day is often smaller than between different days. If the model relies on features describing biofilm characteristics that are tied to specific dates rather than strains across different dates, it may lead to bias and poorer results. We suggest that the features included in rankings_label and rankings_date are compared and that the high-ranking, overlapping features are not considered when explaining the complexity of the system.

Visualisations of the comparisons of the performance of classifiers and the ablation ranking

The results of the performance comparison between the selected classifiers (based on calculations from the classification_all and classification_no_counts_features) and the ablation ranking (ablation_ranking) are visualised to facilitate the interpretation of the results. These are found in the folder “Visualisations”.

8. Inference — predicting biological labels for new images

Inference is the process of using a trained machine learning model to predict biological labels – for example, strain identity – for new, previously unseen images. Unlike the benchmarking described in Section 5, inference does not evaluate model performance; it applies an already trained model to images whose labels are unknown.

Inference requires two things:

1. A trained model. A model must first be built and saved from your training dataset. This is done with the `learning_benchmark_save_models` command described below, which performs the full benchmarking procedure and saves the resulting models for later use. The benchmarking results are therefore available alongside the saved models, allowing you to assess performance on your training data before applying a model to new images.

2. Features calculated from your unknown images. The new images must be passed through the same feature extraction step as the training images, run with the `--unlabeled` flag. Inference operates on this feature table, not on the images directly.

A model can only read the kind of feature table it was trained on. If the model was trained on MicroICS features alone (Workflow B in the Quick guide), supply MicroICS features alone. If it was trained on MicroICS plus external features (Workflow C in the Quick guide), the same external features must be added to the unknown images, generated with the same tool and the same settings. Feature names and column order must be identical.

Before you begin – important preparation steps

File naming: Before running any training command, ensure all images in your training set follow the exact naming convention described in Section 2. Any deviations or extra characters in filenames may cause parsing errors and interrupt the model training process.

Control image splitting (recommended): For optimal inference performance, we recommend manually dividing any control images from treatment experiments into two equal subsets before running this section:

- One subset included in the training set to serve as data augmentation and improve model generalisation
- One subset reserved for inference only to serve as a sanity check on model performance

This split must be performed manually by the user before running the commands below.

6.1. Build and save the models

Run the following command to perform benchmarking and save the trained models for inference:

<code>docker</code>	<code>compose</code>	<code>run</code>	<code>--rm</code>	<code>image</code>	<code>1</code>	<code>datafile.tsv</code>	<code>10</code>
<code>learning_benchmark_save_models</code>							

To include all available algorithms (see Section 5.2 for notes on computational requirements):

```
docker compose run --rm image 1 datafile.tsv 10  
learning_benchmark_save_models --all_learners
```

If you are using a custom configuration file:

```
docker compose --env-file test_paths.txt run --rm image 1 datafile.tsv 10  
learning_benchmark_save_models --all_learners
```

Checking that models were saved correctly: Once this command has completed, navigate to your results folder and look for a subfolder named `saved_models` (or equivalent – confirm the folder name in your version). This folder should contain one saved model file per algorithm that was run. If this folder is empty or missing, the command did not complete successfully and inference cannot proceed.

If you run the learning benchmark again (Section 5) after saving models, the results files will be overwritten. If you wish to keep your benchmarking results, save a copy of the results folder before re-running.

After running this command, check the file `classification_all` in your results folder. This file contains accuracy information for each model based on your training set and will help you identify the best-performing classifier for your dataset. Note that feature rankings are currently provided only for random forest.

6.2. Calculate features on new images

The same feature extraction described in Section 4 must be run on the unknown images, with one difference: the `--unlabeled` flag tells MicroICS that these images have no known label. Nothing is read from the `_st_` field of the filename.

Images are read from `IMAGINE_INFERENCE_INPUTS`, and the resulting feature table `unknown_features.tsv` is written to `IMAGINE_INFERENCE_DATAFILE`.

```
docker compose --env-file test_paths.txt run --rm image 4 unknown_features.tsv  
10 generate_features --unlabeled
```

If you are on Workflow E from the Quick guide, stop here and add your external features to `unknown_features.tsv` before continuing, using the same tool and the same settings as in training.

6.3. Run inference on new images

Once the models have been successfully saved and features from the unknown images extracted, run inference on your new images:

```
docker compose run --rm image 1 unknown_features.tsv 10 inference
```

To run inference using all available algorithms:

```
docker compose run --rm image 1 unknown_features.tsv 10 inference --all_learners
```

If you are using a custom configuration file:

```
docker compose --env-file test_paths.txt run --rm image 1 unknown_features.tsv 10 inference --all_learners
```

6.2. Understanding the inference output

The most important results of the inference are the prediction (majority vote), the probabilities, and the visualisation of the most significant feature values to support interpretation.

The results of the inference by random forest (rf) are contained in different files:

- `inference_summary` – A summary of the inference run, including the model used, the number of classes, and the unique predictions made across all images.
- `rf_predictions` – The predicted biological label for each image. This is the primary output for interpreting which class each new image was assigned to.
- `rf_probabilities` – The confidence score associated with each prediction, indicating how strongly the model associates each image with the predicted class. Scores closer to 1 indicate high confidence; lower scores suggest the image may share structural characteristics with more than one class or represent a phenotype not well represented in the training set.
- `rf_features` – The structural feature values calculated for the new images submitted for inference. These can be used to compare the feature profiles of unknown images with those of the training set.
- `explanations` – A visualisation of SHAP values showing how individual features contributed to each prediction for each class in the training set. This output supports biological interpretation by identifying which structural descriptors influenced the model's decision for each image.

Results of inference for other models, if activated by `--all_learners`, have 'rf' in the above filenames replaced by the name of the other tested models.

Output file explanations:

inference_summary

- `Model`: The name of the model used for inference; "rf" denotes random forest.
- `Num_predictions`: The total number of images for which inference was performed.

- **Unique_predictions:** The number of distinct predicted classes in the inference dataset.

rf_features

The feature values calculated for each image in the dataset used for prediction.

rf_predictions

- **Sample_name:** The identifier or name of the image on which inference was performed.
- **Prediction:** The inferred class or strain name predicted by the model based on the features extracted from the image.

rf_probabilities

For each class present in the training set, a probability is assigned representing the likelihood that the unknown sample belongs to that class. These probabilities sum to 1 and are calculated based on the proportion of random forest trees voting for each class. This probability provides a measure of confidence in the prediction. When multiple classes have similar probabilities, the prediction is ambiguous, and the user should examine feature-level details via the provided visualisations to better understand the model's decision.

Explanations

For every image in the inference dataset, plots derived from SHAP (SHapley Additive exPlanations) values are provided. SHAP values are a method used to explain the output of machine learning models. They quantify the contribution of each feature to the model's prediction. A higher SHAP value for a feature indicates that this feature has a stronger positive influence on predicting a specific class, while a negative value means the feature decreases the likelihood of that prediction. SHAP values help clarify which characteristics of an input (in this case, image features) drive the model's decisions, making the "black box" of machine learning more interpretable.

For each image a separate plot is generated for each possible class present in the training data, enabling comparison across all relevant categories. These plots are particularly useful in two scenarios:

- When two classes have similar prediction probabilities, allowing identification of specific features that may have led to confusion or misclassification.
- When misclassification occurs, where the inferred class changes and differs from the true class (e.g., from strain L628 to strain L1764), the plots help identify which features contributed most to this switch compared to the original class.

rf_summary_plot

This plot shows how multiple structural features influenced the model's predictions across all submitted images. Each dot represents a single image, positioned and coloured according to the SHAP value of a given feature for that image. The predicted class shown for each dot is the class assigned to that image by majority vote, so images predicted to belong to different classes appear together on the same plot, each reflecting its own individual prediction. Each dot represents the SHAP value of a single feature for a single image, positioned along the x-axis to indicate the strength and direction of that feature's influence on the predicted class. The predicted class for each image is determined individually by majority vote, so a single plot displays all images together, with each dot reflecting the prediction made for that specific image.

Dot colour indicates the actual value of that feature in the image: red dots represent high feature values, and blue dots represent low feature values. This allows you to see not only which features are important, but also in which direction they influence the prediction – for example, whether a high fractal dimension pushes the model towards one class or another. Clusters of similarly coloured dots on the same side of the plot indicate that the model consistently relies on the magnitude of those features when making predictions, rather than simply their presence or absence. This visualisation therefore highlights the combined contribution of multiple structural features to each classification decision and can help guide biological interpretation of the results.

rf_feature_importance

For each feature calculated, the strength of its relationship to the target is indicated, with larger values showing that the feature was more important in predicting strain, and vice versa. This is similar to the file `rankings_label` provided for the training set, and here it is provided for the inference set.

rf_shap_values

For each image in the inference set, the table lists all extracted features with their corresponding SHAP values, indicating the contribution and importance of each feature to the model's prediction.

Recommended workflow to interpret machine learning inference results

To avoid confusion, we use the following terms consistently throughout the following description:

- **Class:** The category or label that the machine learning model assigns to an image (e.g., a predicted identity of a strain).
- **Strain:** The actual biological category or type of the sample being analysed, such as a specific bacterial strain. This represents the true class in experimental terms.
- **Treatment:** The experimental condition or manipulation applied to a biological sample (e.g., a drug or environmental change) before imaging. Multiple images from replicated treatments are used to assess model predictions.

Experimental design

We recommend performing multiple technical replicates and independent experiments on different dates for each treatment and control condition. This approach increases robustness and enables more reliable interpretation of results.

Control experiments integration

Control images should be divided and included in both the training and inference sets as sanity controls. This helps validate the model's performance and supports interpretation.

Aggregation of inference results

After inference, collect all technical and independent repetitions of the same treatment condition (Figure 2). Construct a matrix where inferred classes are counted per treatment, as shown in Table 1. This table records the number of images predicted for each class (strain) from the training set, helping to identify the predominant inferred class per treatment condition. The data for this table are from the file `rf_predictions` (Figure 2).

sample_name	prediction	
05--03--2024--s--Lm--st--L1764--p--B02--pos003--tm--24--ch--Syto9--z--21--treat--CTRL	L1764	
05--03--2024--s--Lm--st--L1764--p--B02--pos004--tm--24--ch--Syto9--z--21--treat--CTRL	L1764	
05--03--2024--s--Lm--st--L1764--p--B02--pos005--tm--24--ch--Syto9--z--21--treat--CTRL	L455	
05--03--2024--s--Lm--st--L1764--p--B03--pos001--tm--24--ch--Syto9--z--21--treat--CTRL	L1764	
05--03--2024--s--Lm--st--L1764--p--B03--pos002--tm--24--ch--Syto9--z--21--treat--CTRL	L1323	
05--03--2024--s--Lm--st--L1764--p--B03--pos003--tm--24--ch--Syto9--z--21--treat--CTRL	L1764	
05--03--2024--s--Lm--st--L1764--p--B04--pos001--tm--24--ch--Syto9--z--21--treat--CTRL	L1764	
05--03--2024--s--Lm--st--L1764--p--B05--pos002--tm--24--ch--Syto9--z--21--treat--CTRL	L394	
05--03--2024--s--Lm--st--L1764--p--B05--pos004--tm--24--ch--Syto9--z--21--treat--CTRL	L1764	
05--03--2024--s--Lm--st--L1764--p--B05--pos005--tm--24--ch--Syto9--z--21--treat--CTRL	L1764	
05--03--2024--s--Lm--st--L1764--p--B06--pos001--tm--24--ch--Syto9--z--21--treat--CTRL	L455	
05--03--2024--s--Lm--st--L1764--p--B06--pos004--tm--24--ch--Syto9--z--21--treat--CTRL	L1764	
05--03--2024--s--Lm--st--L1764--p--B06--pos005--tm--24--ch--Syto9--z--21--treat--CTRL	L1764	

Figure 2. Screenshot of the rf_predictions output file in TSV format, which can be opened using Excel.

Table 1. Summary of image counts per class for each treatment condition.

This table shows the number of images classified as belonging to each class (strain) for repeated experiments under the same condition (combination of strain and treatment). Each row represents one treatment condition with multiple replicates. The numbers indicate how many images the model assigned to each class.

strain	treatment	voted class							
		L1764	L455	L1323	L394	L628	ATCC 19115	L1823	L634
L1764	control	19	2	1	3	0	0	0	0
L1764	melon extract coat	46	0	0	4	0	0	0	0
L1764	milk coat	46	0	0	2	0	0	2	0
L1764	tatar steak extract coat	50	0	0	0	0	0	0	0
L628	control	0	0	0	0	24	0	1	0
L628	melon extract coat	21	14	0	7	5	0	2	0
L628	milk coat	42	0	0	4	0	0	4	0
L628	tatar steak extract coat	47	0	0	1	0	0	2	0

The class with the highest count in a given row is considered the predominant inferred class for that treatment condition. This aggregation identifies the overall prediction across multiple images and replicates, increasing confidence in the inference.

Using prediction probabilities

Since individual images may vary, each inference is associated with a probability score indicating the model's confidence in the assigned class. These probabilities are provided in rf_probabilities (Figure 3).

- High probabilities indicate strong confidence in the prediction.
- Probabilities around or below 0.6 require careful examination, especially if the model's probabilities for control classes are similarly close; this may indicate ambiguous or

difficult cases. Low probabilities often mean the model assigned the closest available class in the training set, but with low confidence due to dissimilarities.

- We recommend compiling probability values for all repetitions of each treatment in an Excel table and calculating summary statistics such as mean, median, standard deviation, minimum, maximum, and percentiles. This provides insight into the central tendency and variability of the model's confidence per treatment, highlighting any outliers that could skew averages. The average probability then serves as a guide for interpretation.

	A	B	C	D	E	F	G	H	I
1		19115	L1323	L1764	L1823	L394	L455	L628	L634
2	05-03-2024-s-Lm-st-L1764-p-B02-pos003-tm-24-ch-Syto9-z-21-treat-CTRL	0.00041	0.013219	0.855892	0.004681	0.108014	0.012493	0.004702	0.000589
3	05-03-2024-s-Lm-st-L1764-p-B02-pos004-tm-24-ch-Syto9-z-21-treat-CTRL	0.001855	0.061754	0.687851	0.017926	0.135934	0.064304	0.025095	0.005279
4	05-03-2024-s-Lm-st-L1764-p-B02-pos005-tm-24-ch-Syto9-z-21-treat-CTRL	0.008751	0.078372	0.203608	0.04543	0.228666	0.341288	0.061868	0.032017
5	05-03-2024-s-Lm-st-L1764-p-B03-pos001-tm-24-ch-Syto9-z-21-treat-CTRL	0.001355	0.051938	0.523286	0.015452	0.351344	0.031956	0.023348	0.001321
6	05-03-2024-s-Lm-st-L1764-p-B03-pos002-tm-24-ch-Syto9-z-21-treat-CTRL	0.076983	0.506541	0.06176	0.034767	0.077356	0.121135	0.063011	0.058447
7	05-03-2024-s-Lm-st-L1764-p-B03-pos003-tm-24-ch-Syto9-z-21-treat-CTRL	0.000304	0.010446	0.90777	0.005873	0.050421	0.012033	0.010258	0.002896
8	05-03-2024-s-Lm-st-L1764-p-B04-pos001-tm-24-ch-Syto9-z-21-treat-CTRL	0	0.041062	0.748993	0.007711	0.169848	0.02058	0.011806	0
9	05-03-2024-s-Lm-st-L1764-p-B05-pos002-tm-24-ch-Syto9-z-21-treat-CTRL	0.019718	0.129244	0.297091	0.039861	0.329502	0.112944	0.048037	0.023603
10	05-03-2024-s-Lm-st-L1764-p-B05-pos004-tm-24-ch-Syto9-z-21-treat-CTRL	0.002362	0.081238	0.410088	0.025042	0.371392	0.067717	0.037753	0.00441
11	05-03-2024-s-Lm-st-L1764-p-B05-pos005-tm-24-ch-Syto9-z-21-treat-CTRL	0.008554	0.156446	0.310189	0.032211	0.301204	0.122132	0.057134	0.012129
12	05-03-2024-s-Lm-st-L1764-p-B06-pos001-tm-24-ch-Syto9-z-21-treat-CTRL	0.008373	0.07835	0.130138	0.05785	0.111157	0.336464	0.254194	0.023475
13	05-03-2024-s-Lm-st-L1764-p-B06-pos004-tm-24-ch-Syto9-z-21-treat-CTRL	0.003759	0.086755	0.284825	0.03906	0.276273	0.254748	0.041953	0.012627
14	05-03-2024-s-Lm-st-L1764-p-B06-pos005-tm-24-ch-Syto9-z-21-treat-CTRL	0	0.030838	0.771651	0.00382	0.171032	0.009224	0.013238	0.000197

Figure 3. Screenshot of the rf_probabilities output file in TSV format, viewable in Excel. Column headers include the inferred image names and the names of each class from the training set.

Feature importance analysis

For deeper contextual interpretation, analyse the key features driving the model's decisions:

- Extract the SHAP values for each feature from the rf_shap_values file, which lists SHAP values for all features calculated during feature extraction for all images in the inference set.
- Positive SHAP values indicate features pushing the model toward the predicted class; negative values indicate features working against it.
- If true class labels are known for some examples they can be used to identify correctly and incorrectly classified examples.
- Examine rf_feature_importance for information on the entire dataset for inference.

Evaluating predicted versus expected classifications

In inference experiments, the true biological identity of the images is always known to the researcher – for example, you know that a particular set of images shows strain L628 grown under food extract treatment. When the model predicts a different class for these images – for example, classifying L628 as L1764 — this does not necessarily indicate a model error. Rather, it suggests that the biofilm structure of the treated sample has shifted sufficiently to resemble a different strain, which is itself a biologically meaningful finding.

To explore this further, you can combine the top 15 most important feature rankings with the feature values from the rf_features file to compare the structural characteristics of images that were predicted as expected with those that were predicted as a different class:

- Prediction matches known identity – the biofilm structure is consistent with the strain's typical phenotype under the given conditions.

- Prediction differs from known identity – the biofilm structure has shifted towards another class, indicating treatment-induced or condition-dependent structural reorganisation. Examining the feature values for these images can reveal which structural properties changed and in which direction, providing quantitative insight into the nature of the phenotypic shift.

This approach allows you to identify which structural features are associated with class switching and to characterise the magnitude and direction of structural changes in biological terms – for example, whether a treatment caused biofilms to become more dispersed, less complex, or lower in density relative to the training strains.

9. MicroICS implementation examples

7.1. How do you use the pipeline if you want to link biofilm features to properties other than the strain name?

The easiest way is to replace the string after `_st_` in the image name with the desired property. For example, to associate biofilm features with clinical relevance, replace the string after `_st_` (strain name) with `CI` (clinically relevant) or `CNI` (clinically not relevant) based on the epidemiology of the strain. Then run the pipeline as described. For the latter example, the renaming of the image would look like this:

The original name is: `13042023_s_Lm_st_L628_p_C06_pos001_tm_24_ch_Syto9_z_21` and then we replace the `_st_` with `CI`, which means that strain L628 is clinically relevant to get the image name: `13042023_s_Lm_st_CI_p_C06_pos001_tm_24_ch_Syto9_z_21`.

The same applies to strain L1764, which is not clinically relevant:

The original name `13062023_s_Lm_st_L1764_p_G05_pos002_tm_24_ch_Syto9_z_21` becomes `13062023_s_Lm_st_CNI_p_G05_pos002_tm_24_ch_Syto9_z_21`

7.2. Using the pipeline to create images with reduced number of layers and determining whether the number of layers in a 3D image affects classifier performance

The number of layers in the 3D images can be reduced with the command from the pipeline:

```
docker compose run --rm imagine datafile.tsv reduce_layers <initial number of layers>
```

To reduce the layers from 21 to 11, you can use the following example:

```
docker compose run --rm imagine 1 datafile.tsv 10 reduce_layers 21
```

This command removes every other layer from bottom to top to create a 3D image with half of the original layers. By successively performing the reduce-layers task (21, 11, 6, 3), we can reduce the number of layers down to the bottom layer. Then use images with a reduced number of layers to run the pipeline as described above.

10. Feature descriptions to help with interpretation

A set of quantitative features is extracted from each 3D image (Table 2). The 3D images here consist of several layers, e.g., we have a 3D image consisting of 21 layers (Figure 4). Understanding that 3D images consist of several sub-images is crucial for understanding how calculations are performed.

Image analysis typically involves some pre-processing steps to distinguish the background from the foreground and to aid in object recognition within the image. As these procedures require human input, we aim to avoid them. Generally, the input for feature calculation is a raw image; however, for feature counts (number of bacteria), the pixel intensity values in the raw image are also normalised. “Normalisation” for feature count is performed by subtracting each pixel intensity value from the average pixel intensity of that layer and then dividing by the difference between maximum and minimum pixel intensity $(x - \text{average}) / (\text{max} - \text{min})$. This adjusts the range of pixel values in an image to a standard range, here between 0 and 1. If the feature name in the data file contains “Norminten”, the image has been normalised; if the raw image was used as input, it has not.

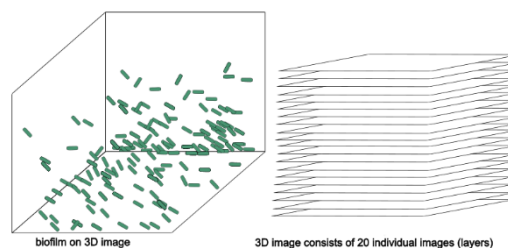


Figure 4. Schematic representation of input images. Green rods on the left image represent bacteria within the biofilm.

All features provide a value for each image but are calculated in two different ways: a) the feature is computed immediately for the whole image or b) the layers are first considered individually and then a value per image is obtained by aggregating the layer values by computing the mean (mean), median (med), minimum (min), maximum (max), standard deviation (std), variance (var) and quantiles (q10, q25, q75 and q90). If the feature is aggregated in the data file, this is noted as `_mean`, `_med`, `_min`, `_max`, `_std`, `_var`, `_q10`, `_q25`, `_q75`, `_q90` at the end of the feature's name. In addition, we also introduce normalised values of features that are calculated for each layer as described later.

Below you will find descriptions of the features. Table 2 lists the abbreviations from the data file, the full names of the features, the processing steps and brief explanations.

Morphological features:

These features capture geometric aspects of the biofilm components.

“Biovolume” is the most commonly used feature in the biofilm community, where the number of pixels belonging to the biofilm is counted and then multiplied by the voxel size (here $0.13 \times 0.13 \times 0.5 \mu\text{m}$). A threshold is set before counting the pixels and only pixels above this threshold are counted. We generate multiple intensity thresholds, resulting in many biovolume features that differ only by the threshold, so the algorithm can capture the values relevant for classification. This feature is calculated for the image as a whole and not per layer.

The “count” features give the number of bacteria on each layer. For this, a threshold is applied and then the biofilm components (here bacteria) are segmented using the connected components approach from Scipy Multidimensional Image Processing (`scipy.ndimage`). The objects are then counted for each layer and the counts per layer are aggregated to obtain one value per image. We generate multiple intensity thresholds, resulting in many count features

that differ only by the threshold value, so that the algorithm can capture the relevant values for classification. The normalized values of the counts are calculated as described earlier. Although counts intuitively reflect the amount of bacteria, they are based on thresholding and only represent bacteria detected above that threshold, which may not capture all bacteria optimally.

“Substratum Relative Coverage”: Percentage of surface covered by biofilm in the 1st layer (surface of the well) of the image.

The statistics-based intensity features:

The feature “globalMean” is extracted from the entire 3D image by calculating the average value of all pixel intensity values in the 3D image and is not derived layer by layer (Figure 2).

A set of intensity-based features labeled “maxdiffs”, “mindiffs”, “mdiffs” is calculated from the differences of the average pixel intensity values between two consecutive layers, i.e., the average value of the first layer is subtracted from the average value of the second layer and so on until 19 differences are obtained (Figure 5). Then the maximum, minimum and median differences are selected and labelled as “maxdiffs”, “mindiffs”, “mdiffs” and given as individual values for each image.

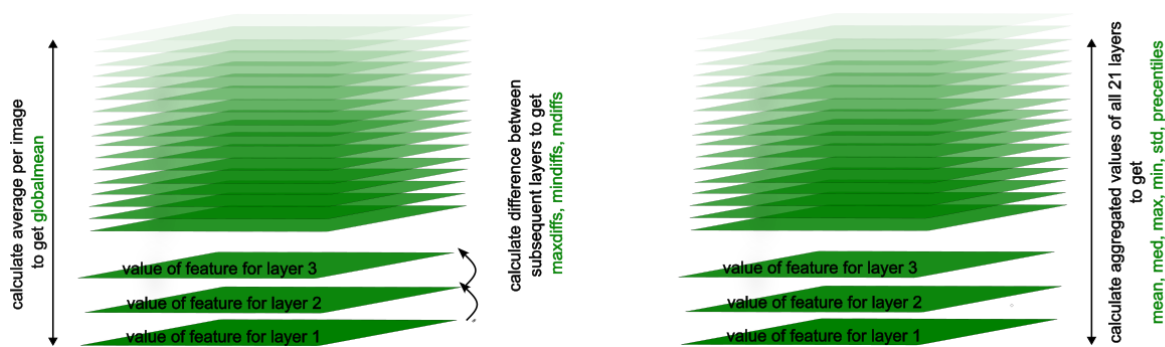


Figure 5. Schematic representation of calculation of intensity-based features.

The features “diff” and “diffNormalized” are calculated by subtracting the maximum pixel intensity value from the minimum pixel value for each layer separately. Consequently, we obtain several values for a feature (one per layer), which we combine into a single value by aggregation. In addition, we also introduce normalized features that are calculated for each layer, where the values are normalized by dividing the feature value for the individual layer by the sum of the values calculated for that layer. For example, the maximum difference in pixel intensity on layer 1 is divided by the sum of the maximum differences calculated for layers 1, 2, 3, 4 up to 21. Normalized features are indicated by the presence of “Normalized” in the feature name, e.g. “diffNormalized”.

Another set of intensity-based features labeled “mean”, “med”, “max”, “min” and “std” are calculated by computing the mean, median, maximum, minimum or standard deviation of the pixel intensity values for each layer, and these values are then aggregated. Normalized values are also calculated for these features by calculating the proportion for each layer.

Texture features:

Texture features provide a description of an image that contains information about the spatial distribution of pixel intensity values.

Void fraction “minprop” is calculated by counting the number of pixels in a layer whose pixel intensity value is below the average pixel intensity value in that layer. This feature is calculated separately for each layer and then aggregated for the entire image as described above. Normalized values are available as “MinPropNormalized”. This feature describes the empty space of the image that is not occupied by bacteria.

The “Homogeneity” is calculated based on the co-occurrence matrix, which represents the distribution of changes in the pixel intensity values of neighboring pixels in the X, Y and Z directions. The formula for calculating homogeneity has already been described¹. It measures the similarity of image structures that are spatially close to each other: a higher homogeneity indicates a more homogeneous image structure. This feature is calculated for the image as a whole.

“ThicknessThreshold” is calculated by tracking the pixel intensity value of each pixel from the first layer (substrate) in height (along the normal to the substrate) until the pixel intensity value reaches 0. The number of the layer where the pixel intensity value reaches 0 is marked, and then the average value of these layers (heights) is calculated. One value is calculated per image.

“roughnessThreshold” is calculated by measuring the height differences for consecutive pixels calculated for “ThicknessThreshold” and then calculating the average. It reflects the height differences of the biofilm.

“Spreading” refers to the spread of the biofilm volume in the 3D image and is calculated as described before². Three different values are calculated: horizontal spreading (in xy-direction), “Sxy”, vertical spreading (in z-direction), “Sz”, and total spreading (in xyz-space).

Large language model (LLM) based features (GPT features)

“GPT volume” refers to the volume of the image layer occupied by the biofilm, where the number of bacteria is multiplied by the voxel size and then aggregated per image.

“GPT fractal dimension” captures the complexity of the geometric patterns on each image layer and is then aggregated into a single value.

“GPTContrast”, “GPTDissimilarity”, “GPTHomogeneity”, “GPTEnergy”, “GPTCorrelation” and “GPTASM” were calculated using the Graycomatrix method. In this method, a gray-level co-occurrence matrix (GLCM) is created by counting how often a pixel with intensity i occurs in a specific spatial relationship to a pixel with intensity j . The correlation measures, for example, how strongly pairs of grey levels vary together, which indicates predictable spatial patterns in the image.

“GPT correlation”: Similar to the Pearson correlation coefficient, the correlation value is calculated based on the correlations between the values in the graycomatrix. If the correlations are high, this indicates that certain grey level pairs occur in a predictable manner.

“GPT Contrast”: The contrast highlights the difference between gray-level values. Larger intensity differences contribute more to the sum, so a higher contrast value indicates greater variability in the intensities of neighbouring pixels.

“GPT dissimilarity”: Similar to contrast, but it is calculated from absolute differences rather than a squared difference. Dissimilarity also captures local variations in grey levels, but its contribution grows more linearly compared to the quadratic term of contrast.

“GPT homogeneity”: It is calculated in a very similar way to non-GPT homogeneity, but here it is first calculated per layer and then summarised per image by aggregation. However, a high homogeneity indicates a more uniform and less varied texture.

“GPT energy”: Energy is another way of expressing the uniformity or regularity of texture. It provides an indication of how consistent the structure of the biofilm is at the microscopic level.

Table 2. List and descriptions of biofilm features calculated by the MicroICS pipeline.

The image can be analysed as a raw image without adjustments or the pixel intensity values can be normalized, which is referred to as norminten. The pixel intensity values in the raw image are normalized per layer by subtracting the average pixel intensity value of the individual layer by the sum of average pixel intensity values for all layers to obtain values between 0 and 1.

All features in the data file are aggregated by calculating the mean, median, maximum, minimum and quartiles 10, 25, 75 and 90, standard deviation and variance across the image layers. If the name of the feature contains "SUMMARY", this means that it is an aggregated value of the feature, and the aggregation type is written after the underscore at the end of the feature name _mean, _med, _min, _max, _q10, _q25, _q75, _q90, _std, _var.

Morphological features:

Feature name in the manuscript	Feature name in the datafile provided by the pipeline	Input	Value normalization	Calculation per _	Description of feature	Threshold
Counts (threshold < n)*	counts(inten<threshold-SUMMARYCustomAlgos.tsv	raw image	no	layer	The number of bacteria in each layer of the 3D image is determined by first applying the threshold value, then segmenting the individual bacteria, and finally counting them. The raw image or an image whose intensity values have been normalized is used as input. For the "normalized" feature, the counts per layer are normalized as a proportion of the sum of all values * 100 (to obtain percentages) per value.	Range from 0.01 to 0.2 increments for 0.01
Counts (normalized intensity, threshold < n)*	counts(Norminten<threshold-SUMMARYCustomAlgos.tsv	intensity normalised on raw image	yes	layer		Range from 0.01 to 0.2 increments for 0.01
Counts (normalized per layer, threshold < n)*	counts(inten<thresholdNormalized-SUMMARYCustomAlgos.tsv	raw image	yes	layer		Range from 0.01 to 0.2 increments for 0.01
Counts (normalized intensity, normalized per layer, threshold < n)*	counts(Norminten<thresholdNormalized-SUMMARYCustomAlgos.tsv	intensity normalised on raw image	yes	layer		Range from 0.01 to 0.2 increments for 0.01
Biovolume	BioVolumeThr-SUMMARYCustomAlgos.tsv	raw image	no	image	The biofilm volume is calculated by counting the number of pixels belonging to the biofilm after thresholding and multiplying this by the voxel size, which represents the physical dimensions of each pixel in height, width, and length (in micrometres). This gives the total volume occupied by the biofilm in the 3D image. Only one volume value is given per image, and the aggregated values remain consistent.	Range from 0.01 to 0.29 increments for 0.01
GPT volume	GPTVolume-SUMMARYCustomAlgos.tsv	raw image	no	layer	The GPT volume is similar to the biovolume described above, but here the biovolume is first determined layer by layer after thresholding, and then the values of these layer counts are aggregated for each image.	Fixed value determined for each image
Normalized GPT volume	GPTVolumeNormalized-SUMMARYCustomAlgos.tsv	raw image	yes	layer		Fixed value determined for each image
Substratum relative coverage	SubstratumRelativeCoverage-SUMMARYCustomAlgos.tsv	raw image	no	image	Percentage of the surface of the well (1st layer of the image) covered with biofilm.	Fixed value determined for each image

*Feature "counts": Counts are a unique feature that captures variations in counts of bacteria within biofilm by changing the threshold used to detect individual bacteria. It is an ensemble of features known to be correlated with each other and useful in machine learning to capture all aspects of thresholding and counting.

Statistics-based intensity features:

Feature name in the manuscript	Feature name in the datafile provided by the pipeline	Input type	normalization	Calculation per _	Description of feature	Threshold
Global mean	globalMean-SUMMARYDiffGlobal.tsv	raw image	no	image	The average pixel intensity of the entire 3D image reflects the overall brightness of the biofilm. A single value is calculated for each image, whereby all aggregated values are identical.	no
Layer-wise global maximum**	maxdiffs-SUMMARYDiffGlobal.tsv	raw image	no	image	The average pixel intensities are calculated for each layer of the 3D image, and the difference between the average of each layer and the next is calculated. Only the maximum difference is given, which represents the largest change in brightness between the layers and extends the understanding of the global mean.	no
Layer-wise global minimum**	mindiffs-SUMMARYDiffGlobal.tsv	raw image	no	image	The difference between the average intensities of consecutive layers is calculated in a similar way, but only the minimum difference is given to highlight areas of little or no change, which may correspond to stable biofilm regions or the background.	no
Layer-wise global mean**	mdiffs-SUMMARYDiffGlobal.tsv	raw image	no	image	The mean difference between the intensities of successive layers is calculated and reported and reflects the overall uniformity or variability of brightness in the vertical dimension of the biofilm.	no
/	eigen-SUMMARYDiffGlobal.tsv	raw image	no	/	A value representing the differences in the overall 3D image is noted but is not currently calculated.	no
Intensity differences**	diff-SUMMARYCustomAlgos.tsv	raw image	no	layer	The difference between maximum and minimum pixel intensity within each layer is calculated and aggregated, where: (a) the mean aggregation indicates the image contrast, b) the median reflects the typical intensity, c) the standard deviation indicates the variability of the contrast between the layers, and d) the quantiles describe the distribution skewness of the contrast between the layers.	no
Normalized intensity differences**	diffNormalized-SUMMARYCustomAlgos.tsv	raw image	yes	layer	The differences between the maximum and minimum intensities per layer are normalized as a percentage of the total max-min sum across all layers and then aggregated to characterize the relative contrast distribution.	no
Mean intensity	mean-SUMMARYCustomAlgos.tsv	raw image	no	layer	The mean pixel intensity is calculated for each layer and then averaged across the layers, representing the average brightness per layer; higher values indicate denser or brighter regions and vice versa.	no
Normalized mean intensity	meanNormalized-SUMMARYCustomAlgos.tsv	raw image	yes	layer	The mean intensity on each layer is normalized as a percentage of the total sum of mean intensities per image before averaging across layers.	no
Median intensity	med-SUMMARYCustomAlgos.tsv	raw image	no	layer	The median pixel intensity is calculated for each layer, and then the median values are aggregated across all layers.	no
Normalized median intensity	medNormalized-SUMMARYCustomAlgos.tsv	raw image	yes	layer	The median intensity per layer is normalized as a percentage of the total sum of the median intensities per image and then aggregated across layers.	no
Maximum intensity	max-SUMMARYCustomAlgos.tsv	raw image	no	layer	The maximum pixel intensity is calculated per layer and aggregated, highlighting localized bright spots such as clumps or active cells.	no

Feature name in the manuscript	Feature name in the datafile provided by the pipeline	Input type	normalization	Calculation per _	Description of feature	Threshold
Normalized maximum intensity	maxNormalized-SUMMARYCustomAlgos.tsv	raw image	yes	layer	The maximum intensity per layer is normalized as a percentage of the total maximum per image and then aggregated.	no
Minimum intensity	min-SUMMARYCustomAlgos.tsv	raw image	no	layer	The minimum pixel intensity per layer is calculated and aggregated to capture the darkest or least dense biofilm regions; in combination with other statistics, it shows biofilm gaps, heterogeneity and spatial structure.	no
Normalized minimum intensity	minNormalized-SUMMARYCustomAlgos.tsv	raw image	yes	layer	The minimum intensity per layer is normalized as a percentage of the total minimum sum per image before aggregation.	no
Standard deviation of intensity	std-SUMMARYCustomAlgos.tsv	raw image	no	layer	The standard deviation of pixel intensities per layer is calculated and aggregated; low values indicate uniform intensity (low contrast), while high values indicate high contrast and high variability.	no
Normalized standard deviation of intensity	stdNormalized-SUMMARYCustomAlgos.tsv	raw image	yes	layer	The standard deviation per layer is normalized as a percentage of the total standard deviation sum per image and then aggregated.	no

****Layer wise features:**

Layer wise features capture how much the intensity fluctuates between the layers and how features differ across the volume.

Texture features:

Feature name in the manuscript	Feature name in the datafile provided by the pipeline	Input type	normalization	Calculation per _	Description of feature	Threshold
Void fraction (min Prop)	minProp-SUMMARYCustomAlgos.tsv	raw image	no	layer	The number of pixels with intensity below the average pixel intensity is counted for each image. Min prop refers to dark areas, such as empty regions not occupied by biofilm.	no
Normalized void fraction	minPropNormalized-SUMMARYCustomAlgos.tsv	raw image	yes	layer		no
Homogeneity	Homogeneity-SUMMARYCustomAlgos.tsv	raw image	no	image	Homogeneity is calculated using the co-occurrence matrix, which captures the distribution of intensity changes between neighbouring pixels in the X, Y and Z directions (after Beyenal et al. 2004, doi: 10.1016/j.mimet.2004.08.003). It measures how similar neighbouring image objects are, with higher values indicating greater uniformity. One value is given per image, and the aggregated values are identical.	no
Thickness	ThicknessThreshold=0.01-SUMMARYCustomAlgos.tsv	raw image	no	image	Pixel intensities are tracked vertically from the first image layer, recording for each pixel the height at which the intensity drops to zero. The average of these heights is reported as a single image value using the same aggregation.	Range from 0.01 to 0.29 increments for 0.01
Roughnes	RoughnessThreshold=threshold-SUMMARYCustomAlgos.tsv	raw image	no	image	The average height difference between neighbouring pixels at a given layer thickness is calculated and reflects the roughness of the biofilm surface. One value per image is given, with uniform aggregation.	Range from 0.01 to 0.29 increments for 0.01
Horizontal spreading	SpreadingHorizontal-SUMMARYCustomAlgos.tsv	raw image	no	image	Three distributions (Dx, Dy, Dz) of the x, y, and z coordinates of the biofilm pixels are created, as described by Mueller et al. (2006, doi: 10.1186/1472-6785-6-1), and the variances for each axis are calculated. Horizontal spreading is calculated by summing up variances in x and y. Vertical spreading (in z direction) equals the variance of z. Total spreading (in z direction) is calculated by summing up variances in x, y and z.	no
Total spreading	SpreadingTotal-SUMMARYCustomAlgos.tsv	raw image	no	image		no
Vertical spreading	SpreadingVertical-SUMMARYCustomAlgos.tsv	raw image	no	image		no
GPT fractal dimension	GPTFractalDim-SUMMARYCustomAlgos.tsv	raw image	no	layer	The fractal dimension reflects the complexity and irregularity of the surface or boundary of the biofilm and quantifies this complexity at different scales; higher values indicate more complex and branched structures rather than smooth shapes.	no
Normalized GPT fractal dimension	GPTFractalDimNormalized-SUMMARYCustomAlgos.tsv	raw image	yes	layer		no
GPT homogeneity	GPTHomogeneity-SUMMARYCustomAlgos.tsv	raw image	no	image	This feature is calculated similarly to non-GPT homogeneity, but is first computed for each layer and then aggregated for each image; higher homogeneity indicates a more uniform and consistent texture.	no

The following features were calculated by Graycomatrix method, which constructs a grayscale co-occurrence matrix (GLCM) by calculating how many times a pixel with intensity value (grayscale) i occurs in a given spatial relationship to a pixel with value j .

Feature name in the manuscript	Feature name in the datafile provided by the pipeline	Input type	normalization	Calculation per	Description of feature	Threshold
GPT contrast	GPTContrast-SUMMARYCustomAlgos.tsv	raw image	no	image	Contrast emphasises the difference between grey-level values. Larger differences in intensity contribute more to the sum, so a higher contrast value indicates greater variability in the intensities of neighbouring pixels.	no
GPT correlation	GPTCorrelation-SUMMARYCustomAlgos.tsv	raw image	no	image	The correlation value is calculated similarly to the Pearson correlation coefficient, using the correlations between values in the graycomatrix. High correlations indicate that certain grey-level pairs occur in a predictable manner.	no
GPT dissimilarity	GPTDissimilarity-SUMMARYCustomAlgos.tsv	raw image	no	image	Similar to contrast, but it is calculated from absolute differences instead of a squared difference. Dissimilarity also captures local variations in gray levels, but its contribution grows more linearly compared to contrast's squared term. Dissimilarity is opposite to homogeneity.	no
GPT energy	GPTEnergy-SUMMARYCustomAlgos.tsv	raw image	no	image	Energy is another way to express the uniformity or orderliness in the texture. It is often used because it is on a [0,1] scale if P is normalized.	no